FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Daniel Marçal Gonçalves

Produto: **Entre Cartas (Between Letters)**
https://github.com/danielmarcalgoncalves/entre-cartas

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 5
3.3 Fluxo de uma requisição 6
3.4 Fluxo de assinatura 7
3.5 Modelo de dados do domínio 8
3.6 Segurança 10
3.7 Observabilidade, CI/CD e documentação da API 11
3.8 Camadas IaaS, PaaS e SaaS do projeto 11
4 Quadro de entregas 12
5 Atividades realizadas 12
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 12
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 14
5.3 Atividade 9 — Fork e plano de negócio 16
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 18
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 21
6 Considerações finais 25
7 Declaração de autoria 25

1 Introdução

A disciplina de **Introdução à Computação em Nuvem (ISW055)**, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na Fatec Pompeia, tem como premissa formar engenheiros de software capazes de projetar, construir e manter aplicações reais em nuvem. Enquanto o primeiro bimestre concentrou-se no desacoplamento gradual de responsabilidades em microsserviços sobre um domínio didático (o **Catálogo de Filmes Tom Hanks**), o segundo bimestre elevou o desafio ao nível de engenharia de produto: converter a fundação técnica construída em um **Software as a Service** (SaaS) autoral, dotado de tese de negócio, modelo de monetização recorrente, limites operacionais por plano, governança administrativa e alta disponibilidade em produção.

Dessa evolução nasceu o **Entre Cartas** (**Between Letters**), um santuário digital de escrita íntima, diários encadernados (**journaling**), **scrapbook** interativo e correspondência editorial. O produto ataca uma dor real vivenciada por estudantes universitários, leitores e jovens adultos (18 a 35 anos, representados pela persona Clara Menezes) que buscam desacelerar da hiperconectividade das redes sociais por meio da escrita reflexiva. Hoje, esse público enfrenta um dilema claro: ou gasta entre R$ 60,00 e R$ 250,00 mensais em cadernos físicos artesanais e papelaria — que não possuem busca textual, backup em nuvem nem proteção contra perda ou leitura indevida por terceiros —, ou recorre a ferramentas corporativas frias e assépticas como Notion e Google Docs, que destroem a atmosfera literária e agravam o bloqueio criativo diante da página em branco.

Para resolver esse problema com excelência comercial — atendendo diretamente ao objetivo central proposto pelo professor de conceber um SaaS real pronto para ser vendido no mercado —, optou-se deliberadamente por construir uma aplicação de altíssima complexidade técnica e riqueza de experiência de usuário, indo muito além de um CRUD convencional. O **Entre Cartas** implementa uma escrivaninha interativa (**My Desk**) com cadernos em couro dotados de motor próprio de paginação contínua automática, camada de **scrapbook** visual (com upload de fotografias para bucket dedicado no MinIO, molduras **polaroid/vintage**, adesivos vetoriais, controle de rotação, escala, camadas `z-index` e pilha de **undo/redo**), editor epistolar com papéis e tintas históricas, estante tridimensional de pastas (**My Shelf**) com lixeira postal recuperável (**soft delete**), motor de prompts de inspiração literária e proteção de borda com Cloudflare. Na retaguarda, o sistema opera um fluxo completo de faturamento com Stripe Checkout, Customer Billing Portal, tratamento de falhas de renovação (`past_due`) com janela de graça e um painel administrativo executivo com cálculo de MRR estimado, inspeção individual de contas e auditoria em tempo real via Redis Streams.

Cumpre destacar, com total transparência acadêmica, que a decisão de desenvolver um produto com esse grau de profundidade arquitetural, refinamento visual e capacidade real de escalabilidade horizontal demandou ciclos intensos de engenharia, testes de integração entre contêineres e ajustes finos de segurança no ambiente do Portainer. Em razão dessa complexidade superior e do compromisso em entregar um SaaS verdadeiramente comercializável e robusto — e não apenas um protótipo simplificado —, algumas entregas intermediárias registraram atraso pontual no cronograma. Solicita-se, portanto, que a avaliação considere a maturidade técnica, a riqueza funcional e a escalabilidade atingidas pelo sistema em produção. Este relatório documenta, na Seção 3, a arquitetura completa que sustenta o produto no ar e, nas Seções 4 e 5, o quadro consolidado e as fichas comprobatórias das Atividades 7 a 11.

2 Metodologia

A trajetória técnica deste segundo bimestre iniciou-se no repositório público original do primeiro bimestre (https://github.com/danielmarcalgoncalves/filmecc), onde foram desenvolvidas e validadas a Atividade

7 (integração de cobrança recorrente com Stripe Checkout e Webhook idempotente) e a Atividade 8 (mapeamento das camadas IaaS, PaaS e SaaS e engenharia reversa completa no arquivo `DOCUMENTACAO.md`). Em seguida, na Atividade 9, realizou-se a migração integral do histórico de versionamento (`git clone --bare` seguido de `git push --mirror`) para o novo repositório dedicado ao produto comercial (https://github.com/danielmarcalgoncalves/entre-cartas). Esse procedimento preservou intactos todos os hashes SHA-1, autores, datas e horários desde o início do semestre, garantindo rastreabilidade contínua entre o projeto precursor e o SaaS final desenvolvido nas Atividades 10 e 11.

A infraestrutura de desenvolvimento e produção baseia-se em contêineres Linux orquestrados via **Docker Compose** e gerenciados remotamente pelo **Portainer** na máquina virtual da disciplina, respondendo publicamente sob o domínio https://daniel-marcal-isw055.lapps.studio. A arquitetura adota um build **multi-stage** no `Dockerfile` principal: o primeiro estágio (`frontend-builder`, sobre `node:20-slim`) compila a aplicação **Next.js 16 / React 19 / TypeScript** em artefatos estáticos otimizados (`out/`), enquanto o segundo estágio (`runtime`) acopla esses artefatos ao servidor **Node.js / Express**, que atua simultaneamente como servidor web e API Gateway/BFF na porta `3000`. Na rede interna isolada do Docker (`entre-cartas_default`), operam de forma desacoplada o microsserviço de identidade e RBAC (`auth-service`), o microsserviço de auditoria (`log-service`), o banco relacional **MariaDB 10.11**, o banco em memória **Redis 7 Alpine** (operando com estrutura **Redis Streams**) e o servidor de objetos **MinIO** compatível com AWS S3.

Para assegurar rigor probatório na prestação de contas, todas as datas e horários informados no quadro de entregas (Seção 4) e nas fichas individuais (Seção 5) foram extraídos diretamente dos metadados imutáveis do Git por meio do comando `git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad"` e confrontados com as páginas públicas de commits e com as **issues** formais de entrega abertas no GitHub com menção explícita ao professor (`@siriani`). Cada ficha reúne o link direto verificável, a captura de tela da evidência de entrega exibindo repositório, hash e carimbo de data/hora, e as capturas de tela da funcionalidade operando no ambiente real.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O **Entre Cartas** opera sob uma arquitetura de microsserviços conteinerizada com persistência poliglota (**Polyglot Persistence**), na qual apenas o contêiner principal (`app`) é exposto ao tráfego externo através do proxy reverso do domínio. Todos os demais serviços de autenticação, auditoria, banco de dados relacional, mensageria em memória e armazenamento de objetos comunicam-se exclusivamente por resolução DNS interna na rede privada `bridge` do Docker (`entre-cartas_default`), eliminando exposição desnecessária de portas sensíveis na máquina virtual.

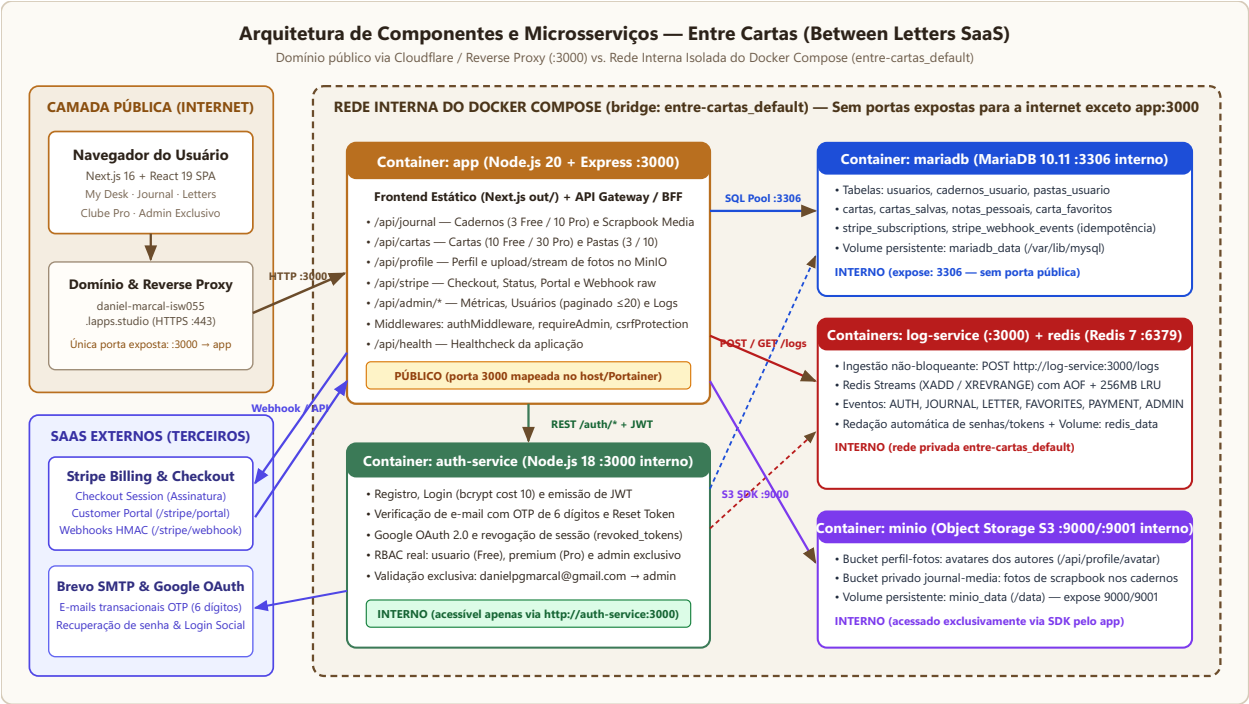


Figura 1: Diagrama de componentes e microsserviços do SaaS Entre Cartas em produção

Componente	Responsabilidade no Entre Cartas	Tecnologia / Imagem	Público?
`app`	Serve o frontend estático (Next.js `out/`), landing page, escrivaninha, estante e atua como API Gateway/BFF (`/api/cartas`, `/api/journal`, `/api/profile`, `/api/stripe`, `/api/admin/*`, `/api/health`) aplicando cotas e RBAC.	Node.js 20 + Express + Next.js 16 (`node:20-slim` multi-stage)	Sim (domínio `:3000`)
`auth-service`	Microsserviço de identidade: cadastro, hash de senha (`bcrypt`), confirmação por OTP de 6 dígitos, login JWT, Google OAuth 2.0, recuperação de senha, revogação de tokens e resolução de papéis RBAC (`usuario`, `premium`, `admin`).	Node.js 18 + Express (`node:18-alpine`)	Não (rede interna `:3000`)
`log-service`	Microsserviço de ingestão (`POST /logs`) e consulta (`GET /logs`) de eventos de negócio, segurança, bloqueios `403` e faturamento com redação automática de campos sensíveis.	Node.js 18 + Express (`node:18-alpine`)	Não (rede interna `:3000`)
`redis`	Armazenamento **append-only** de altíssima vazão dos fluxos de auditoria na estrutura **Redis Streams** (`XADD` / `XREVRANGE` na chave `logs:audit`).	`redis:7-alpine` (AOF + LRU 256 MB)	Não (rede interna `:6379`)
`mariadb`	Persistência relacional ACID de usuários, cadernos (`cadernos_usuario`), pastas (`pastas_usuario`), cartas (`cartas`), carimbos,	`mariadb:10.11` (InnoDB `utf8mb4`)	Não (rede interna `:3306`)

Componente	Responsabilidade no Entre Cartas	Tecnologia / Imagem	Público?
	notas e assinaturas/idempotência Stripe (`stripe_subscriptions`, `stripe_webhook_events`).		
`minio`	Object Storage compatível com S3 para avatares de perfil (`perfil-fotos`) e fotografias de **scrapbook** das páginas de diário (`journal-media`).	`coollabsio/minio:latest`	Não (rede interna `:9000/:9001`)
`Stripe` (externo)	Processamento de checkout de assinatura mensal (`R$ 14,90/mês`), Portal do Cliente (`billingPortal`) e envio assíncrono de webhooks assinados via HMAC-SHA256.	SaaS externo (Stripe API v23)	Webhook → `app`
`Brevo / Google`	Disparo SMTP de códigos OTP de verificação/ recuperação de senha (Brevo) e autenticação federada OAuth 2.0 (Google Cloud).	SaaS externos (SMTP `:587` / HTTPS)	Saída HTTPS/ SMTP

3.2 Onde roda

A aplicação executa na mesma máquina virtual Linux (IaaS da Fatec Pompeia) e na mesma **stack** gerenciada via **Portainer** utilizada desde o início do semestre, agora apontando diretamente para a branch `main` do repositório https://github.com/danielmarcalgoncalves/entre-cartas e publicada no domínio https://daniel-marcal-isw055.lapps.studio. Todos os seis contêineres operam com política de resiliência `restart: unless-stopped`, garantindo recuperação automática em caso de reinicialização do daemon ou falha transitória.

A persistência de estado entre deploys e recriações de contêineres é assegurada por três volumes nomeados gerenciados pelo Docker: `mariadb_data` (montado em `/var/lib/mysql`), `redis_data` (montado em `/data` com `--appendonly yes`) e `minio_data` (montado em `/data`). O serviço `redis` possui limite explícito de memória configurado no comando de inicialização (`--maxmemory 256mb --maxmemory-policy allkeys-lru`), prevenindo exaustão de RAM na máquina virtual compartilhada. A tabela a seguir documenta os contêineres da stack e os **nomes** das variáveis de ambiente parametrizadas no Portainer (cujos valores permanecem estritamente protegidos fora do código-fonte).

Serviço	Porta / Limite / Volume	Nomes das Variáveis de Ambiente Configuradas na Stack
`app`	Host `${RESERVED_PORT}:3000`	`PORT`, `RESERVED_PORT`, `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_CALLBACK_URL`, `MINIO_ENDPOINT`, `MINIO_PORT`, `MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET`, `MINIO_PUBLIC_URL`, `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`, `APP_URL`
`auth-service`	Interna `:3000` (sem porta pública)	`PORT`, `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USER`, `MAIL_PASS`,

Serviço	Porta / Limite / Volume	Nomes das Variáveis de Ambiente Configuradas na Stack
		`MAIL_FROM`, `BREVO_API_KEY`, `BREVO_SENDER_EMAIL`, `BREVO_SENDER_NAME`, `REQUIRE_EMAIL_VERIFICATION`, `APP_URL`, `LOG_SERVICE_URL`
`log-service`	Interna `:3000` (sem porta pública)	`PORT`, `REDIS_URL`, `JWT_SECRET`
`redis`	Interna `:6379` · Limite `256mb` (`allkeys-lru`) · Vol: `redis_data`	Configurado via linha de comando (`redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy allkeys-lru`)
`minio`	Interna `:9000` (API) e `:9001` (Console) · Vol: `minio_data`	`MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`
`mariadb`	Interna `:3306` · Vol: `mariadb_data`	`MYSQL_ROOT_PASSWORD` (mapeado de `DB_PASSWORD`), `MYSQL_DATABASE` (mapeado de `DB_NAME`)

3.3 Fluxo de uma requisição

Para ilustrar como uma ação real do domínio atravessa toda a arquitetura em produção, considera-se a operação central do negócio: **um autor autenticado cria ou salva um volume de caderno pautado na escrivaninha (`PUT /api/journal/notebooks/:clientId`)**:

1. **Navegador → Domínio Público**: Quando o usuário cria um novo caderno ou edita uma página na interface Next.js (`/journal`), o cliente HTTP (`frontend/src/services/api.ts`) dispara uma requisição `PUT https://daniel-marcal-isw055.lapps.studio/api/journal/notebooks/caderno-176307...` enviando o cookie `HttpOnly` `access_token` (e cabeçalho `Authorization: Bearer <token>`) junto ao payload JSON (`nome`, `corCapa`, `paginas`).
2. **Entrada no Contêiner `app` e Proteção de Borda**: O proxy reverso encaminha a chamada à porta `3000` do contêiner `app`. Os middlewares globais validam o cabeçalho `Origin` contra falsificação de requisição (`csrfProtection`, compatível com `X-Forwarded-Host` do Portainer/Cloudflare) e decodificam os cookies.
3. **Autenticação e Avaliação de Papel/Entitlement em Tempo Real (`authMiddleware`)**: O middleware `authMiddleware` (`backend/src/middlewares/auth.js`) verifica a assinatura criptográfica do JWT com `JWT_SECRET` (sincronizado com o `auth-service`), checa na tabela `revoked_tokens` se o token não foi invalidado por logout e executa `evaluateUserEntitlement(pool, usuarioId)` no `MariaDB`. Essa função consulta simultaneamente `usuarios` e `stripe_subscriptions`, determinando em tempo real se o usuário é `usuario` (Free), `premium` (Pro) ou `admin`.
4. **Verificação de Cota por Plano no `MariaDB`**: O controlador `upsertNotebook` (`backend/src/controllers/journalController.js`) verifica se o `clientId` já pertence ao `req.usuarioId`. Caso seja um novo caderno, executa `SELECT COUNT(*) AS total FROM cadernos_usuario WHERE usuario_id = ?`. Se o usuário estiver no plano Free (`usuario`) e já possuir `3` cadernos (ou `10` no plano Pro), a requisição é interrompida com `HTTP 403 Forbidden` padronizado (`code: 'PLAN_LIMIT_REACHED'`), orientando o upgrade para `/clube-pro`.

5. **Persistência Transacional no `MariaDB`**: Estando dentro da cota permitida, o backend sanitiza a estrutura de páginas, fotos de **scrapbook** e **stickers** e executa `INSERT INTO cadernos_usuario (...) VALUES (?, ?, ?, ?, ?)` parametrizado com `req.usuarioId`.
6. **Auditoria Assíncrona no `log-service` + `Redis Streams`**: Em seguida, o backend dispara de forma não-bloqueante (`sendLog` com timeout de 2,5s) um `POST http://log-service:3000/logs` na rede interna do Docker com cabeçalho HMAC `X-Internal-Log-Secret`. O `log-service` grava o evento `JOURNAL_NOTEBOOK_CREATED` no `Redis` via `XADD logs:audit *`.
7. **Resposta ao Cliente**: O contêiner `app` retorna `HTTP 200 OK` com os metadados do caderno salvo, atualizando o indicador de sincronização (`✓ Saved`) na escrivaninha do usuário.

3.4 Fluxo de assinatura

A monetização do **Clube do Remetente Pro** (`R$ 14,90/mês`, exibido como `$14.90/month` na interface internacionalizada) utiliza o fluxo oficial de assinaturas recorrentes da Stripe (`mode: 'subscription'`), garantindo que nenhuma atualização de plano dependa de redirecionamento inseguro no navegador, mas sim de confirmação criptográfica via webhook server-to-server.

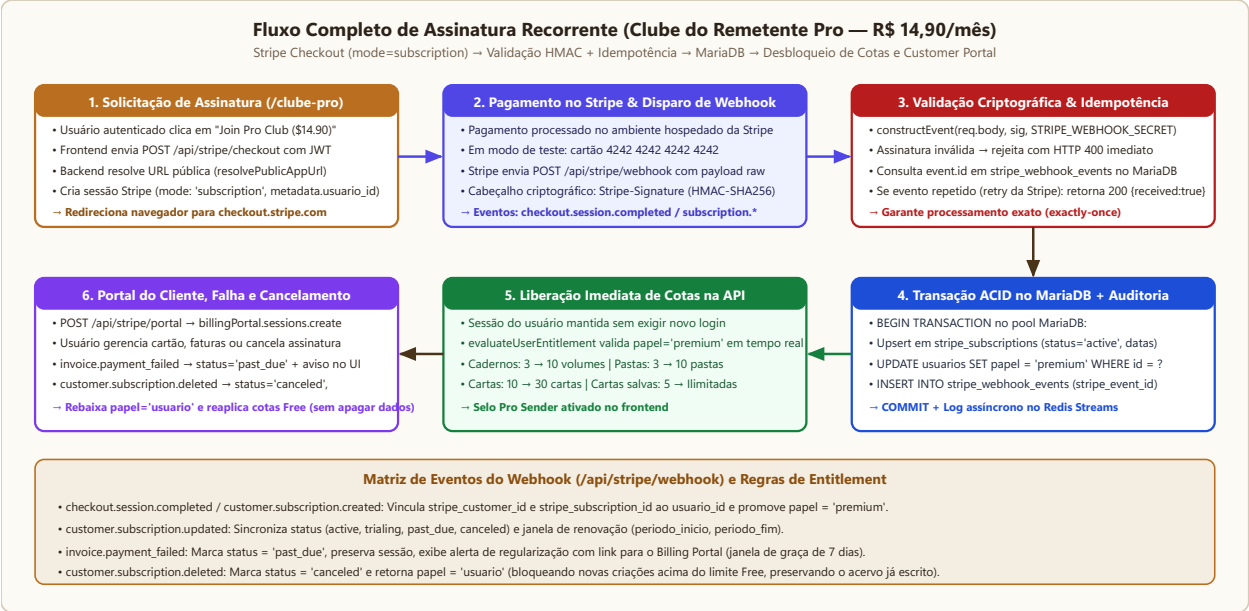


Figura 2: Fluxo completo de assinatura: Stripe Checkout, validação HMAC, idempotência no MariaDB e liberação de cotas

1. **Criação da Checkout Session (`POST /api/stripe/checkout`)**: Ao clicar em **"Join Pro Club for $14.90"** na página `/clube-pro`, o backend autentica o usuário, resolve dinamicamente a origem pública remota (`resolvePublicAppUrl(req)`, evitando URLs `localhost` em produção) e chama `stripe.checkout.sessions.create` em `mode: 'subscription'` com o `STRIPE_PRICE_ID`, anexando `client_reference_id: String(usuarioId)` e `metadata: { usuario_id }`. O usuário é redirecionado ao ambiente seguro da Stripe, onde realiza o pagamento (em modo de teste, com o cartão `4242 4242 4242 4242`).
2. **Recepção e Validação Criptográfica do Webhook (`POST /api/stripe/webhook`)**: A rota `/api/stripe` é registrada no `backend/server.js` **antes** do `express.json()`, preservando o buffer binário bruto (`express.raw({ type: 'application/json' })`). Ao receber o evento da Stripe, o controlador executa `stripe.webhooks.constructEvent(req.body, sig, STRIPE_WEBHOOK_SECRET)`. Qualquer payload sem assinatura HMAC-SHA256 válida é imediatamente rejeitado com `HTTP 400`.
3. **Idempotência Estrita (O que acontece se o mesmo evento chegar duas vezes)**: Antes de aplicar qualquer efeito no banco, o controlador consulta `SELECT id FROM stripe_webhook_events`

`WHERE stripe_event_id = ?`. Caso a Stripe reenvie o mesmo evento por **retry** de rede, o sistema detecta que o `event.id` já consta na tabela, ignora o reprocessamento e responde imediatamente `HTTP 200 { received: true, duplicate: true }`. Se o evento for inédito, a gravação na `stripe_webhook_events`, o **upsert** em `stripe_subscriptions` e a promoção em `usuarios` (`SET papel = 'premium'`) ocorrem dentro de uma única transação ACID (`BEGIN` … `COMMIT`), seguidos de registro de auditoria no `log-service`.

4. **Persistência da Sessão e Liberação Imediata de Limites**: Ao retornar da Stripe para `/clube-pro?checkout=success`, o usuário mantém exatamente a mesma sessão autenticada (via cookie `HttpOnly` e token `Bearer`). O frontend consulta `GET /api/stripe/status` e sincroniza o contexto (`refreshUser()`), liberando instantaneamente na API as cotas expandidas do plano Pro: de **3 para 10 cadernos**, de **3 para 10 pastas** e de **10 para 30 cartas**, além de salvamentos ilimitados no acervo.
5. **Falha de Pagamento (`invoice.payment_failed`) e Cancelamento pelo Portal do Cliente (`POST /api/stripe/portal`)**: Caso a renovação mensal falhe, o webhook `invoice.payment_failed` atualiza `stripe_subscriptions.status = 'past_due'`, preserva a sessão durante a janela de graça de 7 dias e exibe um banner de alerta em `/clube-pro` com botão direto para o **Stripe Customer Billing Portal** (`stripe.billingPortal.sessions.create`). Pelo Portal do Cliente (`billing.stripe.com`), o assinante pode trocar o cartão, consultar faturas ou cancelar a assinatura; quando o cancelamento é efetivado, o webhook `customer.subscription.deleted` marca `status = 'canceled'` e rebaixa `usuarios.papel = 'usuario'`, voltando a aplicar os limites do plano gratuito sem apagar os textos já escritos pelo autor.

3.5 Modelo de dados do domínio

O esquema relacional no **MariaDB 10.11** foi modelado para suportar tanto as entidades de identidade e faturamento recorrente quanto as estruturas completas do domínio literário e epistolar, empregando integridade referencial (`FOREIGN KEY`) e índices dedicados (`idx_usuarios_papel`, `idx_stripe_status`, `uk_usuario_caderno`, `uk_usuario_pasta`).

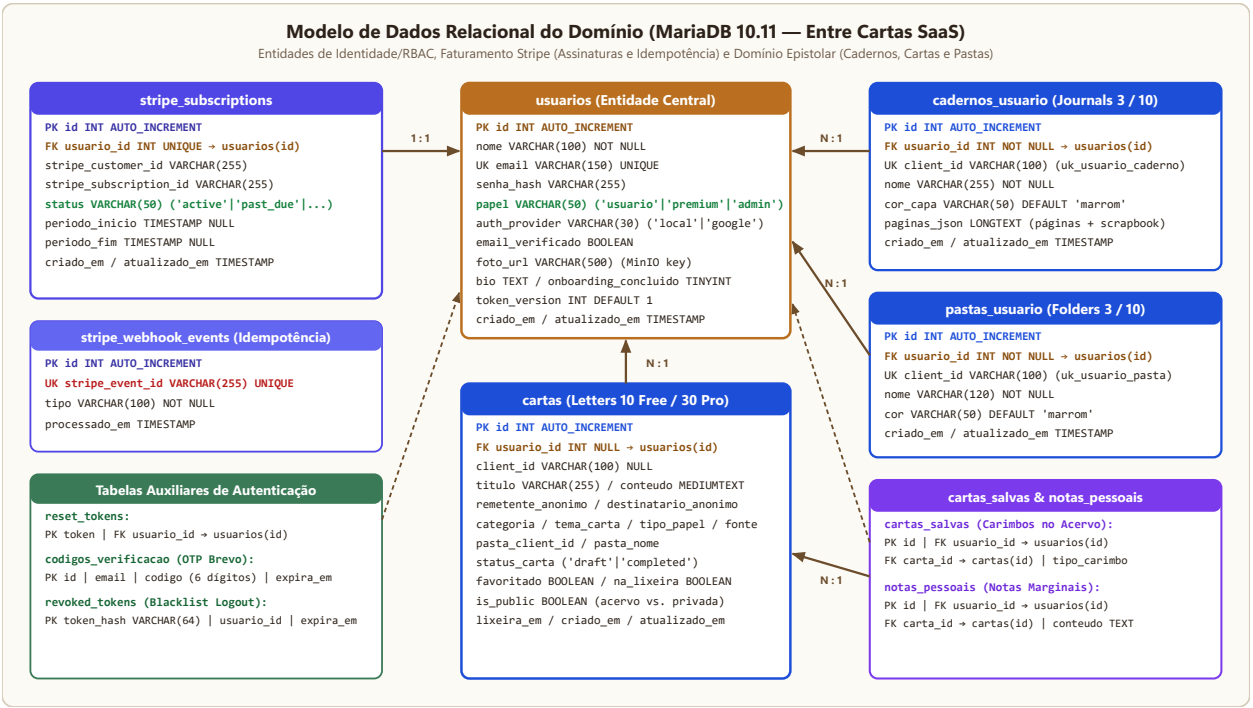


Figura 3: Diagrama Entidade-Relacionamento do domínio Entre Cartas no MariaDB 10.11

Tabela	PK / Chaves Estrangeiras	Colunas Principais e Papel no Domínio
`usuarios`	`PK id` `UK email`	`nome`, `email`, `senha_hash`, `papel` (`'usuario'`, `'premium'`, `'admin'`), `auth_provider` (`'local'`, `'google'`), `email_verificado`, `foto_url` (chave MinIO), `bio`, `onboarding_concluido`, `token_version`, `criado_em`. Liga o usuário ao seu plano e a todos os recursos do domínio.
`stripe_subscriptions`	`PK id` `FK usuario_id → usuarios(id)` (`UNIQUE`)	`stripe_customer_id`, `stripe_subscription_id`, `status` (`'active'`, `'trialing'`, `'past_due'`, `'canceled'`), `periodo_inicio`, `periodo_fim`, `criado_em`, `atualizado_em`. Mantém o vínculo 1:1 entre a conta do usuário e a assinatura na Stripe.
`stripe_webhook_events`	`PK id` `UK stripe_event_id`	`stripe_event_id` (`UNIQUE`), `tipo`, `processado_em`. Garante idempotência no processamento dos webhooks da Stripe.
`cadernos_usuario`	`PK id` `FK usuario_id → usuarios(id)`	`client_id`, `nome`, `cor_capa`, `paginas_json` (`LONGTEXT` contendo páginas escritas, formatação, anexos de foto do MinIO e **stickers**), `criado_em`, `atualizado_em`. Sujeita ao limite de 3 (Free) ou 10 (Pro).
`pastas_usuario`	`PK id` `FK usuario_id → usuarios(id)`	`client_id`, `nome`, `cor`, `criado_em`, `atualizado_em`. Pastas físicas de correspondência na estante do usuário (limite de 3 no plano Free e 10 no plano Pro).
`cartas`	`PK id` `FK usuario_id → usuarios(id)`	`client_id`, `titulo`, `conteudo` (`MEDIUMTEXT`), `remetente_anonimo`, `destinatario_anonimo`, `categoria`, `tema_carta`, `tipo_papel`, `fonte`, `pasta_client_id`, `status_carta` (`'draft'` ou `'completed'`), `favoritado`, `na_lixeira`, `is_public`, `criado_em`. Sujeita à cota de 10 (Free) ou 30 (Pro).

`cartas_salvas` / `carta_favoritos`	`PK id`
	`FK usuario_id → usuarios(id)`
	`FK carta_id → cartas(id)`
		`tipo_carimbo` (`'LIDA'`, `'FAVORITA'`, `'GUARDADA'`), `criado_em`. Registra cartas do acervo público carimbadas e guardadas pelo leitor (cota de 5 no Free e ilimitada no Pro).

`notas_pessoais` / `notas_cartas`	`PK id`
	`FK usuario_id → usuarios(id)`
	`FK carta_id → cartas(id)`
		`conteudo` / `texto`, `criado_em`. Anotações marginais privadas vinculadas às cartas do acervo.

Tabela	PK / Chaves Estrangeiras	Colunas Principais e Papel no Domínio
`codigos_verificacao` / `reset_tokens` / `revoked_tokens`	`PK id` / `PK token` / `PK token_hash`	Suporte ao ciclo de segurança do `auth-service`: códigos OTP de 6 dígitos com expiração temporal, tokens de redefinição de senha e **blacklist** de tokens JWT revogados no logout.

3.6 Segurança

O controle de acesso do **Entre Cartas** implementa RBAC (**Role-Based Access Control**) verificado em banco de dados a cada requisição, combinado com verificação exclusiva de identidade para a governança administrativa. A tabela abaixo detalha as permissões efetivas de cada papel no sistema em produção:

Ação / Recurso no Sistema	`admin`	`premium` (Pro)	`usuario` (Free)
Explorar acervo público e prompts de inspiração literária	Sim	Sim	Sim
Criar e sincronizar cadernos pautados (`/api/journal/notebooks`)	Até 10	Até 10	Até 3 (4º ⟶ `403`)
Criar pastas de correspondência (`/api/journal/folders`)	Até 10	Até 10	Até 3 (4ª ⟶ `403`)
Escrever e selar cartas autorais (`/api/cartas`)	Até 30	Até 30	Até 10 (11ª ⟶ `403`)
Carimbar e guardar cartas do acervo público (`/api/favorites`)	Ilimitado	Ilimitado	Até 5 (6ª ⟶ `403`)
Fazer upload de avatar e fotografias de **scrapbook** no MinIO	Sim	Sim	Sim
Assinar o Clube Pro ou abrir o Portal de Faturamento da Stripe	Sim	Sim (Portal)	Sim (Checkout)
Ler ou modificar cadernos, cartas ou pastas privadas de outro usuário	Não (isolado)	Não (`403/404`)	Não (`403/404`)
Acessar métricas de negócio, listagem paginada de contas, detalhes de uso e logs de auditoria (`/api/admin/*`)	Sim (exclusivo)	Não (`403`)	Não (`403`)

Isolamento de dados por usuário (Anti-IDOR). Todas as consultas e mutações nas tabelas `cadernos_usuario`, `pastas_usuario`, `cartas`, `cartas_salvas` e `notas_pessoais` utilizam **prepared statements** parametrizados (`?`) filtrados estritamente pelo `req.usuarioId` extraído do JWT validado (ex.: `WHERE id = ? AND usuario_id = ?`), jamais confiando em identificadores enviados no corpo da requisição. Além disso, as fotografias de **scrapbook** salvas no bucket privado `journal-media` do MinIO possuem chave prefixada por `u_<usuarioId>/`, e a rota de leitura valida a propriedade do objeto antes de realizar o streaming. No navegador, a troca de conta aciona a limpeza imediata de `localStorage` e `sessionStorage` (`clearAllPrivateBrowserData`), impedindo vazamento de rascunhos entre sessões distintas na mesma máquina.

Proteção administrativa e gestão de segredos. O painel administrativo não utiliza a rota previsível `/admin` (que retorna `404 Not Found`), mas sim a rota privada `/X2fQYLS5UbTYaIchwvwqLqzWqU32ZNoa`, e o middleware `requireAdmin` exige cumulativamente `papel === 'admin'` no MariaDB e e-mail idêntico a `danielpgmarcal@gmail.com`, registrando qualquer tentativa não autorizada como `BLOQUEIO_RBAC_ADMIN_403` no Redis Streams. Por fim, nenhuma chave privada (`JWT_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `DB_PASSWORD`, `BREVO_API_KEY`, `GOOGLE_CLIENT_SECRET`) é versionada no Git: o repositório contém apenas o arquivo `.env.example` com os nomes das variáveis em branco, enquanto os segredos reais residem exclusivamente na configuração de ambiente da stack no Portainer.

3.7 Observabilidade, CI/CD e documentação da API

No eixo de observabilidade, a aplicação implementa o endpoint público de verificação de saúde `GET /api/health` (que valida a disponibilidade do servidor Express na porta `3000`) e um subsistema completo de telemetria administrativa em tempo real (`GET /api/admin/metrics`, `GET /api/admin/users`, `GET /api/admin/users/:id` e `GET /api/admin/logs`), que consolida métricas reais do MariaDB (distribuição de usuários por plano, assinaturas ativas/canceladas/`past_due` e MRR estimado) e eventos operacionais do `log-service` sobre Redis Streams. Além disso, o repositório conta com documentação técnica reversa detalhada em `DOCUMENTACAO.md`. As atividades extras opcionais de pipeline CI/CD via GitHub Actions e especificação interativa via Swagger/OpenAPI não foram incluídas no escopo final para priorizar a robustez funcional, a segurança e a experiência completa do SaaS em produção.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Aplicando os conceitos consolidados na Atividade 8 ao produto final em produção, a arquitetura do **Entre Cartas** distribui-se claramente entre as três camadas clássicas da computação em nuvem:

Camada	Onde aparece no projeto Entre Cartas	Quem gerencia
IaaS	A máquina virtual Linux (provisionada sobre Proxmox/ hipervisor da faculdade), processador (vCPUs), memória RAM física, armazenamento em disco para os volumes persistentes, interface de rede virtual, DNS do domínio `lapps.studio` e roteamento de borda/túnel HTTPS.	Professor (`@siriani`) / Infraestrutura da Fatec Pompeia (com borda Cloudflare)
PaaS	O motor de contêineres **Docker Engine**, a orquestração declarativa via **Docker Compose**, a plataforma visual de gestão de stacks **Portainer** e as imagens base de serviços de plataforma e banco de dados (`mariadb:10.11`, `redis:7-alpine`, `coollabsio/minio`, `node:20-slim` e `node:18-alpine`).	Infraestrutura Docker/Portainer pelo professor; configuração do `docker-compose.yml`, `Dockerfile` e topologia de rede pelo aluno
SaaS	1) **O produto final entregue ao cliente**: a plataforma web **Entre Cartas** (escrivaninha, cadernos, cartas e Clube Pro), pronta para uso no navegador sem instalação local; 2) **Os SaaS externos consumidos via API**: **Stripe** (checkout, assinaturas e portal de faturamento), **Brevo** (e-mails	O código, regras de negócio, limites e banco do **Entre Cartas** pelo aluno; Stripe, Brevo, Google e GitHub pelos

Camada	Onde aparece no projeto Entre Cartas	Quem gerencia
	transacionais OTP), **Google Cloud OAuth** (login social) e **GitHub** (hospedagem de código e issues).	respectivos provedores

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	08/10/2026 20:35	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	19/10/2026 19:46	entregue com atraso
9	Fork e plano de negócio	23/10/2026	11/11/2026 20:13	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	06/11/2026 19:53	entregue
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	13/11/2026 23:16	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	08/10/2026 20:35
Situação	entregue
Evidência	GitHub — commits c5fa977 e 97aa9e1 (08/10/2026) + Issue #6 (filmecc) + prints do checkout e promoção para premium
Link	https://github.com/danielmarcalgoncalves/entre-cartas/commit/97aa9e15a2c6bf0a58a6eff596653bb22292340d

O que foi feito. Implementação do fluxo completo de monetização recorrente em modo de teste utilizando a API oficial da **Stripe** (`stripe` SDK). Foram criadas no MariaDB as tabelas `stripe_subscriptions` (vinculando `usuario_id`, `stripe_customer_id`, `stripe_subscription_id`, `status` e datas de vigência) e `stripe_webhook_events` (garantindo idempotência pelo `stripe_event_id`). No backend (`stripeController.js` e `stripeRoutes.js`), desenvolveu-se o endpoint autenticado `POST /api/stripe/checkout` para geração de sessões `mode: 'subscription'` e o endpoint público `POST /api/stripe/webhook` configurado com `express.raw({ type: 'application/json' })` para verificação criptográfica da assinatura HMAC (`Stripe-Signature`) e promoção automática do usuário para `papel = 'premium'`,

liberando cotas ilimitadas no sistema. O código foi commitado e publicado dentro do prazo em `08/10/2026` às `20:30` (`c5fa977`) e `20:35` (`97aa9e1`), conforme preservado no histórico do repositório.

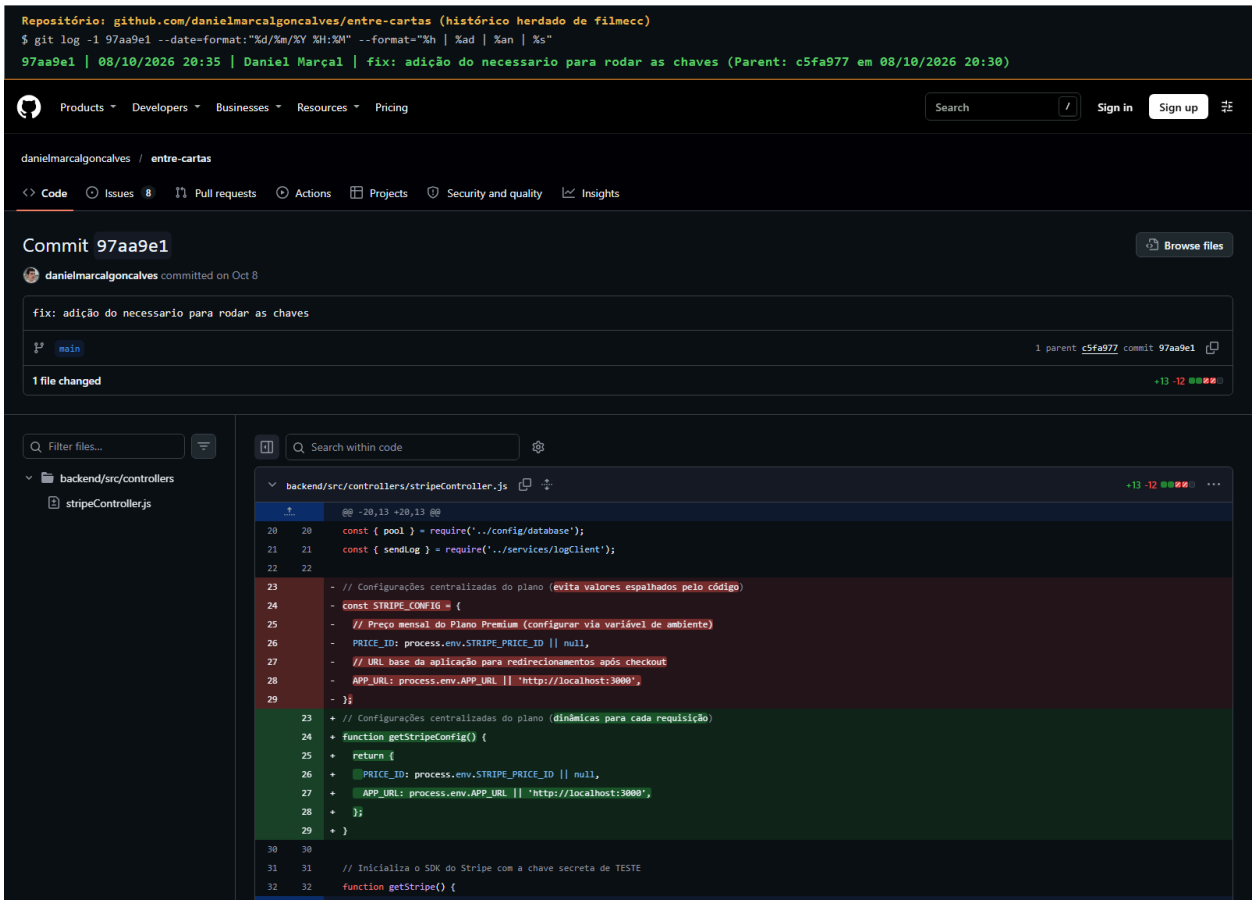


Figura 4: Atividade 7 — Evidência da entrega: commit 97aa9e1 (e parent c5fa977) realizado em 08/10/2026 às 20:35 no GitHub



Figura 5: Atividade 7 — Resultado: tela de checkout da Stripe em modo de teste e confirmação do Plano Premium ativo após processamento do webhook

Dificuldades e como foram resolvidas. O principal desafio técnico consistiu na validação da assinatura criptográfica do webhook da Stripe (`constructEvent`), que falhava quando o middleware global `express.json()` convertia o corpo da requisição em objeto JavaScript antes de atingir a rota do webhook. O problema foi solucionado montando a rota `/api/stripe` no `server.js` antes do parser JSON global e aplicando `express.raw({ type: 'application/json' })` especificamente no endpoint `/webhook`.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	19/10/2026 19:46
Situação	entregue com atraso
Evidência	GitHub — commit 83737a7 + DOCUMENTACAO.md linkado no README + Issue #7 no repositório filmecc
Link	https://github.com/danielmarcalgoncalves/entre-cartas/commit/83737a7d776cf769e667ab9f37f320828a703cc9

O que foi feito. Elaboração de um documento completo de engenharia reversa (`DOCUMENTACAO.md`, com mais de 420 linhas técnicas e diagramas em Mermaid) descrevendo integralmente o sistema em operação sem alterar linhas de código de negócio. O documento estruturou: (1) Especificação Funcional detalhada por perfil de acesso (**Visitante**, **Usuário Comum**, **Assinante Premium** e **Administrador**); (2) Arquitetura Geral do Sistema distinguindo claramente as camadas **IaaS** (VM Proxmox Ubuntu da Fatec), **PaaS** (Docker Compose, Portainer, MariaDB, Redis e MinIO) e **SaaS** (aplicação web e serviços terceiros TMDB, Brevo e Stripe), além de diagramas de sequência; (3) Catálogo de 16 Regras de Negócio extraídas do código no formato **"Se X, então Y"**; (4) Modelo de Dados completo com Diagrama Entidade-Relacionamento (DER); e (5) Mapeamento de todos os endpoints REST. O arquivo foi devidamente referenciado no `README.md`.

Motivo do atraso. A entrega ocorreu 3 dias após a data planejada porque optou-se por produzir uma documentação reversa exaustiva e de nível profissional — inspecionando cada controlador, middleware e tabela para mapear todas as regras de negócio, fluxos de sequência e esquemas relacionais — em vez de entregar um resumo superficial.

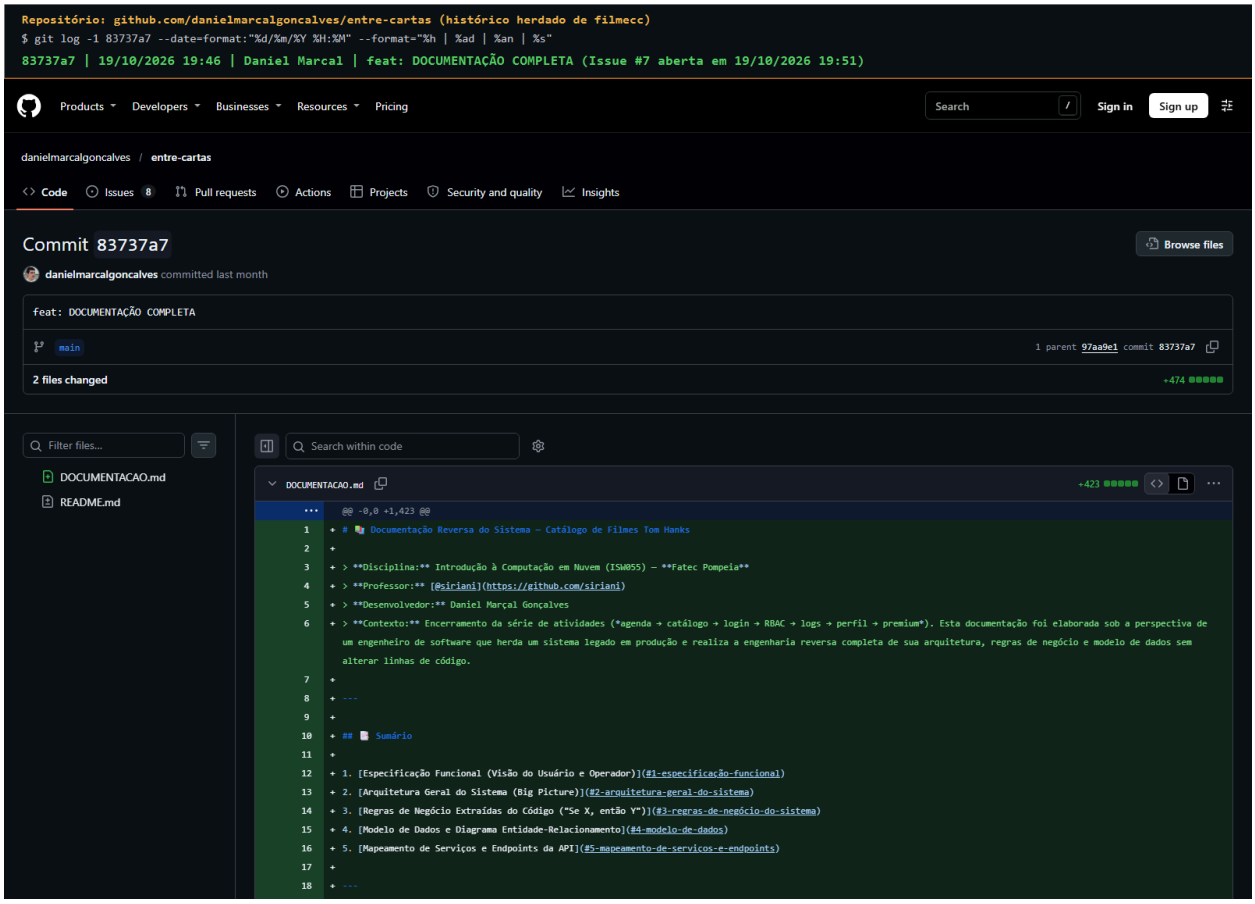


Figura 6: Atividade 8 — Evidência da entrega: commit 83737a7 realizado em 19/10/2026 às 19:46 adicionando DOCUMENTACAO.md e link no README

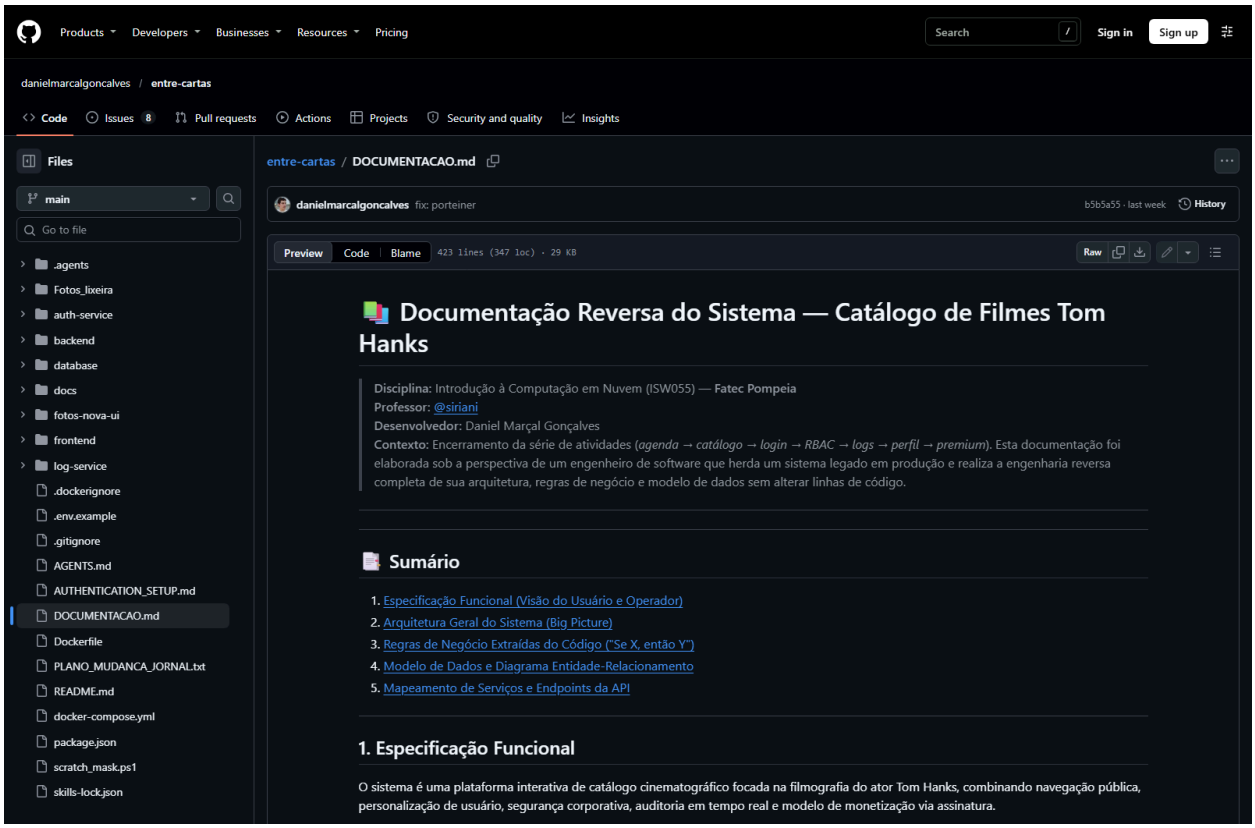


Figura 7: Atividade 8 — Resultado: arquivo DOCUMENTACAO.md renderizado no GitHub com sumário, diagramas de camadas IaaS/PaaS/SaaS e regras de negócio

Dificuldades e como foram resolvidas. A principal dificuldade foi traduzir regras implícitas espalhadas entre middlewares de segurança, microsserviço de autenticação e gatilhos de banco em especificações formais claras para alguém que nunca viu o código. Isso foi resolvido auditando cada rota do Express e organizando as regras por domínio funcional em tabelas e fluxogramas Mermaid verificáveis.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	11/11/2026 20:13
Situação	entregue com atraso
Evidência	GitHub — Issue #7 (@siriani) + commit dc6da84 (iniciado no commit 737e94b em 22/10/2026) + README do produto + prints de commits e issues
Link	https://github.com/danielmarcalgoncalves/entre-cartas/issues/7

O que foi feito. Criação do novo repositório público do produto (https://github.com/danielmarcalgoncalves/entre-cartas) preservando 100% do histórico de commits do repositório anterior (`filmecc`), com os primeiros commits da transformação em **Between Letters / Entre Cartas** realizados já em `22/10/2026 às 14:22` (commit `737e94b`) e `23/10/2026` (commits `0151c5c`, `cc7429a`, `ac7ec05` e `63551f2`). Foi estruturado o plano de negócio completo no `README.md` (consolidado no commit `dc6da84` e formalizado na Issue #7 mencionando `@siriani`), contendo a definição da dor e seu custo real, a persona **Clara Menezes**,

análise comparativa de concorrentes, proposta única de valor, modelo de receita recorrente com Stripe (**Plano Standard Free** vs. **Clube do Remetente Pro — R$ 14,90/mês**), métricas de sucesso (MAU, conversão, MRR e retenção D30), **Lean Canvas** de 9 blocos e o Mapa de Reaproveitamento completo dos microsserviços. Também foram abertas as 6 issues estruturadas do backlog técnico no GitHub.

Motivo do atraso. Embora a migração do repositório e a construção do novo produto tenham começado dentro do prazo (em `22/10/2026`), a formalização final do `README.md` de negócios e a abertura da issue de entrega ocorreram em `11/11/2026` porque dedicou-se um esforço extraordinário de engenharia para conceber um SaaS comercialmente viável e altamente escalável (incluindo motor de cadernos paginados, **scrapbook** interativo e nova identidade visual editorial), alinhando o plano de negócio exatamente às funcionalidades avançadas implementadas.

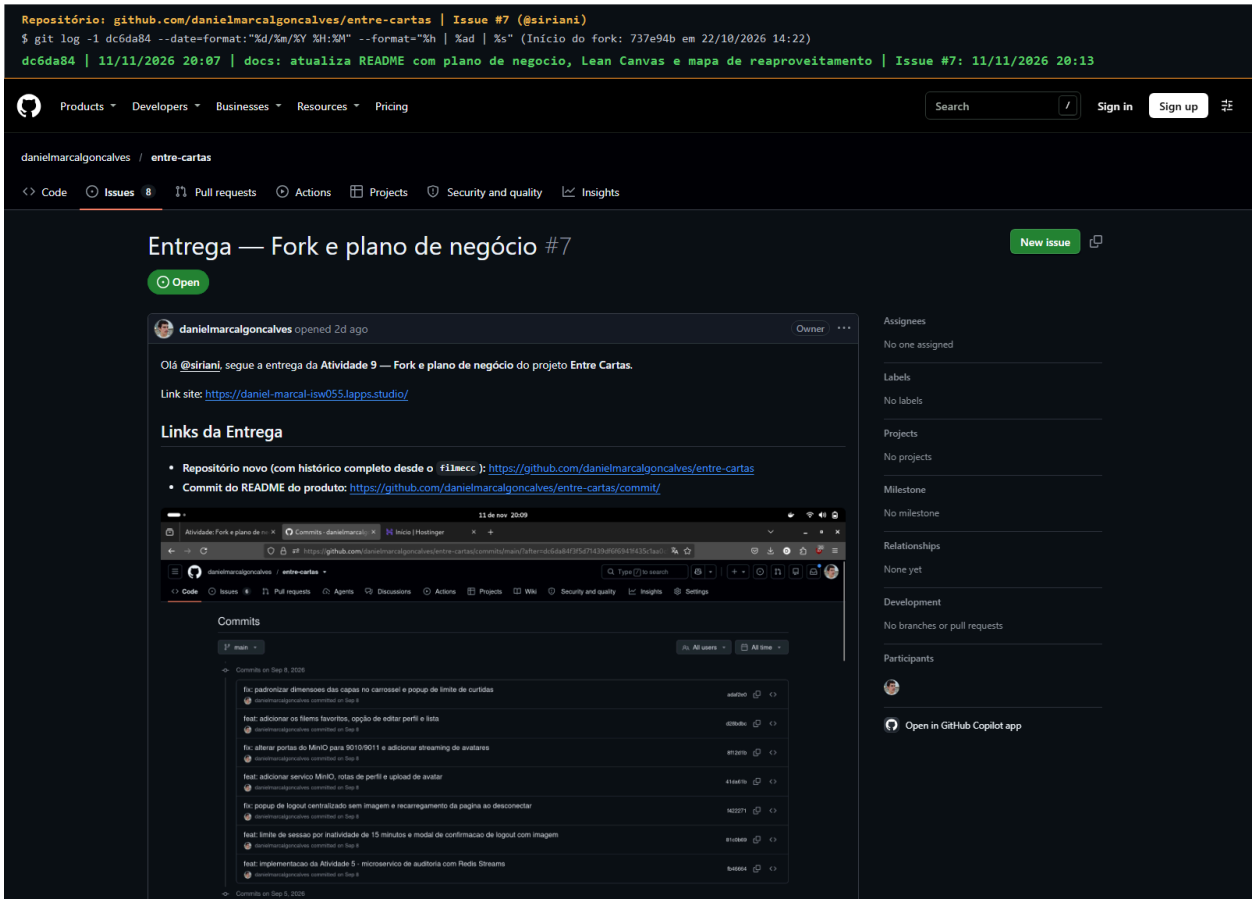


Figura 8: Atividade 9 — Evidência da entrega: Issue 7 mencionando `@siriani` e commit dc6da84 no repositório entre-cartas

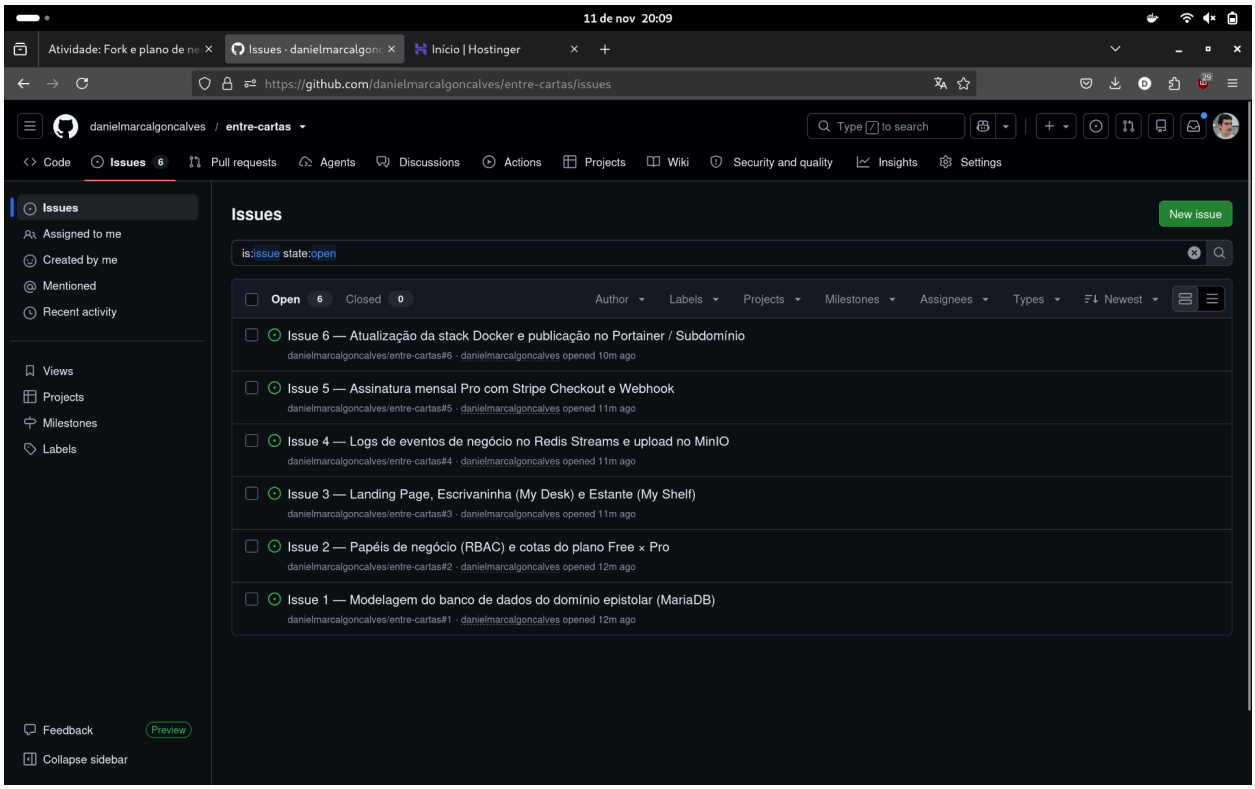


Figura 9: Atividade 9 — Resultado: histórico de commits herdado integralmente do filmecc e lista de 6 issues do backlog abertas no GitHub

Dificuldades e como foram resolvidas. O maior desafio foi desenhar um modelo de negócios em que os limites entre os planos gratuito e pago fizessem sentido orgânico para o usuário final sem descaracterizar a proposta acolhedora do produto. A solução foi calibrar a degustação gratuita em 3 cadernos, 3 pastas e 10 cartas — volume suficiente para criar o hábito semanal de escrita — e posicionar o **Clube do Remetente Pro** (10 cadernos, 10 pastas e 30 cartas por R$ 14,90/mês) como uma alternativa muito mais acessível e segura do que a compra recorrente de cadernos físicos.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	06/11/2026 19:53
Situação	entregue
Evidência	GitHub — commits 50a6538 e 61d1636 (06/11/2026) + estabilização Portainer/ Cloudflare (809b4a1) + URL no ar + prints da landing, journal e logs 403
Link	https://github.com/danielmarcalgoncalves/entre-cartas/commit/50a6538cc67bb6cad57aed5e13da5bbc4d582dc2

O que foi feito. Substituição integral do domínio de filmes pelo domínio literário e epistolar do **Entre Cartas** em toda a stack Docker/Portainer sob o subdomínio https://daniel-marcal-isw055.lapps.studio. No MariaDB e no backend Express, foram implementadas as tabelas e rotas do negócio (`/api/journal/notebooks`, `/api/journal/folders`, `/api/cartas`, `/api/profile`), garantindo isolamento estrito por `usuario_id` em todas as consultas. No frontend Next.js, construiu-se a Landing Page autoral, a página

de introdução interativa (`/explore`), a escrivaninha (**My Desk**), o editor de cadernos pautados com **scrapbook** de fotos no MinIO e **stickers** (`/journal`), o editor de cartas seladas (`/letters`) e a estante com pastas 3D e lixeira postal (`/my-shelf`). O `log-service` com Redis Streams passou a registrar todos os eventos de negócio (`JOURNAL_NOTEBOOK_CREATED`, `CARTA_CRIADA`, `FOLDER_CREATED`) e bloqueios de segurança (`BLOQUEIO_RBAC_ADMIN_403`).

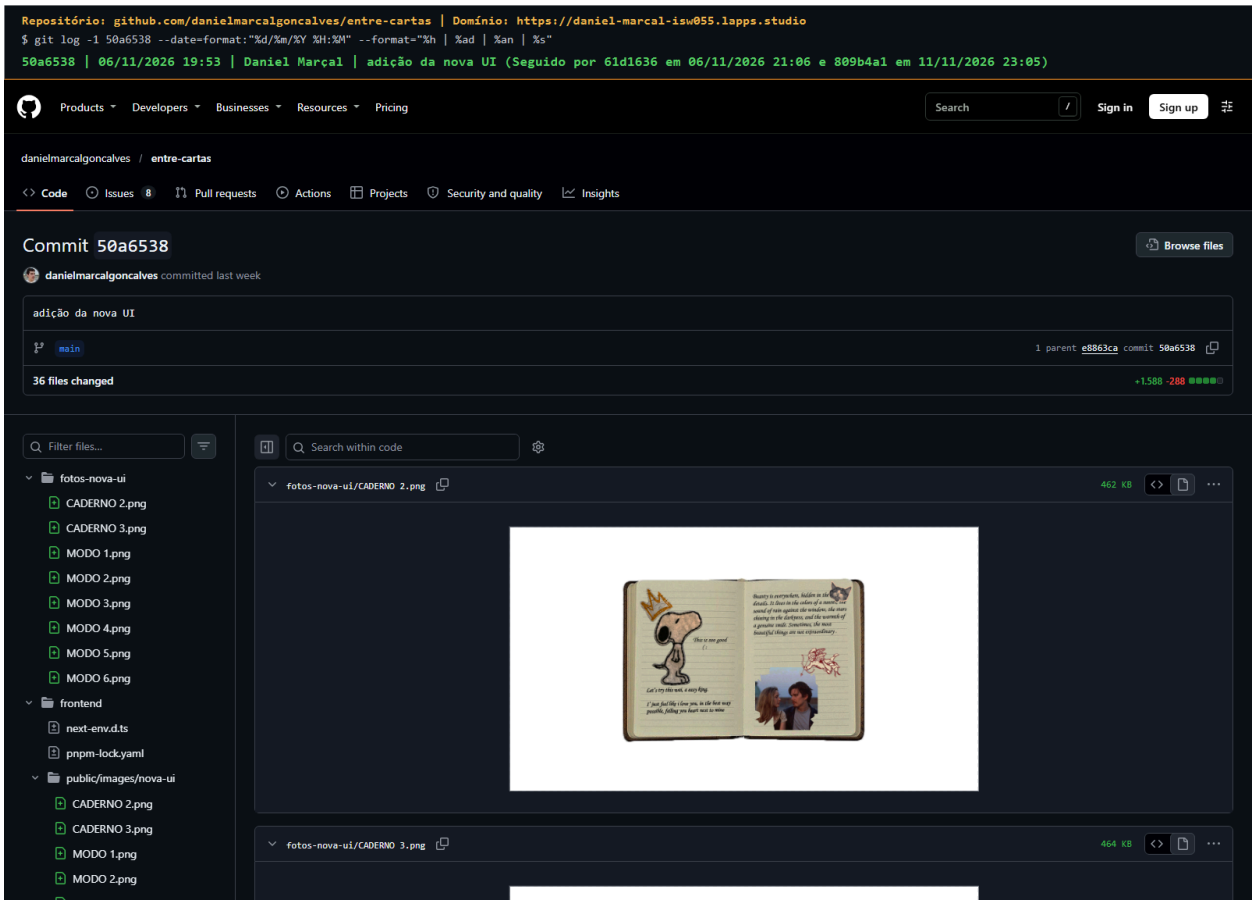


Figura 10: Atividade 10 — Evidência da entrega: commit 50a6538 realizado em 06/11/2026 às 19:53 (seguido por 61d1636 às 21:06) no repositório entre-cartas



Figura 11: Atividade 10 — Resultado: Landing Page e apresentação interativa do Entre Cartas no ar com cadernos e cartas

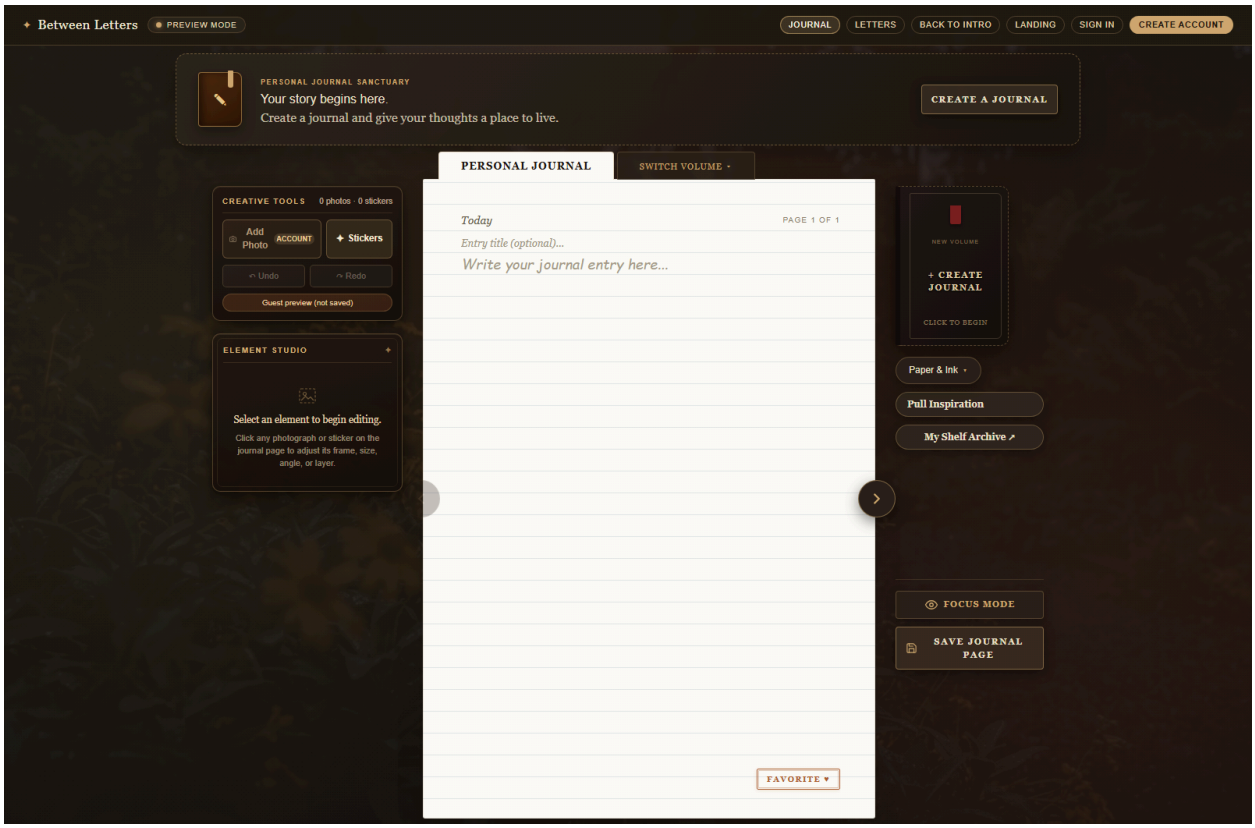


Figura 12: Atividade 10 — Resultado: Editor de Cadernos (Personal Journal) com ferramentas de Scrapbook (fotos no MinIO e stickers) e paginação contínua

Dificuldades e como foram resolvidas. Durante o deploy na stack remota do Portainer, a compilação estática do Next.js (`output: 'export'`) precisava conviver de forma transparente com as rotas da API Express no mesmo contêiner na porta `3000`. O problema foi resolvido estruturando um `Dockerfile` multi-stage que copia o diretório compilado `/app/frontend/out` para `/app/backend/public` e configurando um roteador de fallback no Express que resolve dinamicamente tanto rotas `/api/*` quanto páginas `.html` pré-renderizadas.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	13/11/2026 23:16
Situação	entregue
Evidência	GitHub — Issue #8 (@siriani) + commit d305494 (13/11/2026 23:16) + prints do limite bloqueando, checkout 4242, webhook Pro, Billing Portal e Admin
Link	https://github.com/danielmarcalgoncalves/entre-cartas/issues/8

O que foi feito. Fechamento do ciclo completo do SaaS em produção no domínio https://daniel-marcal-isw055.lapps.studio. Foram consolidados os limites por plano diretamente nos controladores da API (`buildPlanLimitError`): usuários no plano **Standard Free** (`usuario`) são bloqueados com `HTTP 403` ao tentar criar o 4º caderno, a 4ª pasta ou a 11ª carta, recebendo aviso claro e atalho para upgrade. Na página `/clube-pro`, o usuário realiza a assinatura mensal de R$ 14,90 (`$14.90/month`) via **Stripe Checkout** com o cartão de teste `4242 4242 4242 4242`; o webhook assinado (`POST /api/stripe/webhook`) processa os eventos `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted` e `invoice.payment_failed` com idempotência no MariaDB, promovendo o usuário para `premium` sem derrubar sua sessão ativa e liberando imediatamente até 10 cadernos, 10 pastas e 30 cartas. Implementou-se também o **Stripe Customer Billing Portal** (`POST /api/stripe/portal`) para gestão e cancelamento da assinatura, além do **Admin Dashboard** exclusivo para `danielpgmarcal@gmail.com` com métricas reais do MariaDB, cálculo de MRR estimado, listagem paginada de até 20 usuários por requisição, modo de inspeção detalhada de conta e visualizador filtrável de logs do Redis Streams.

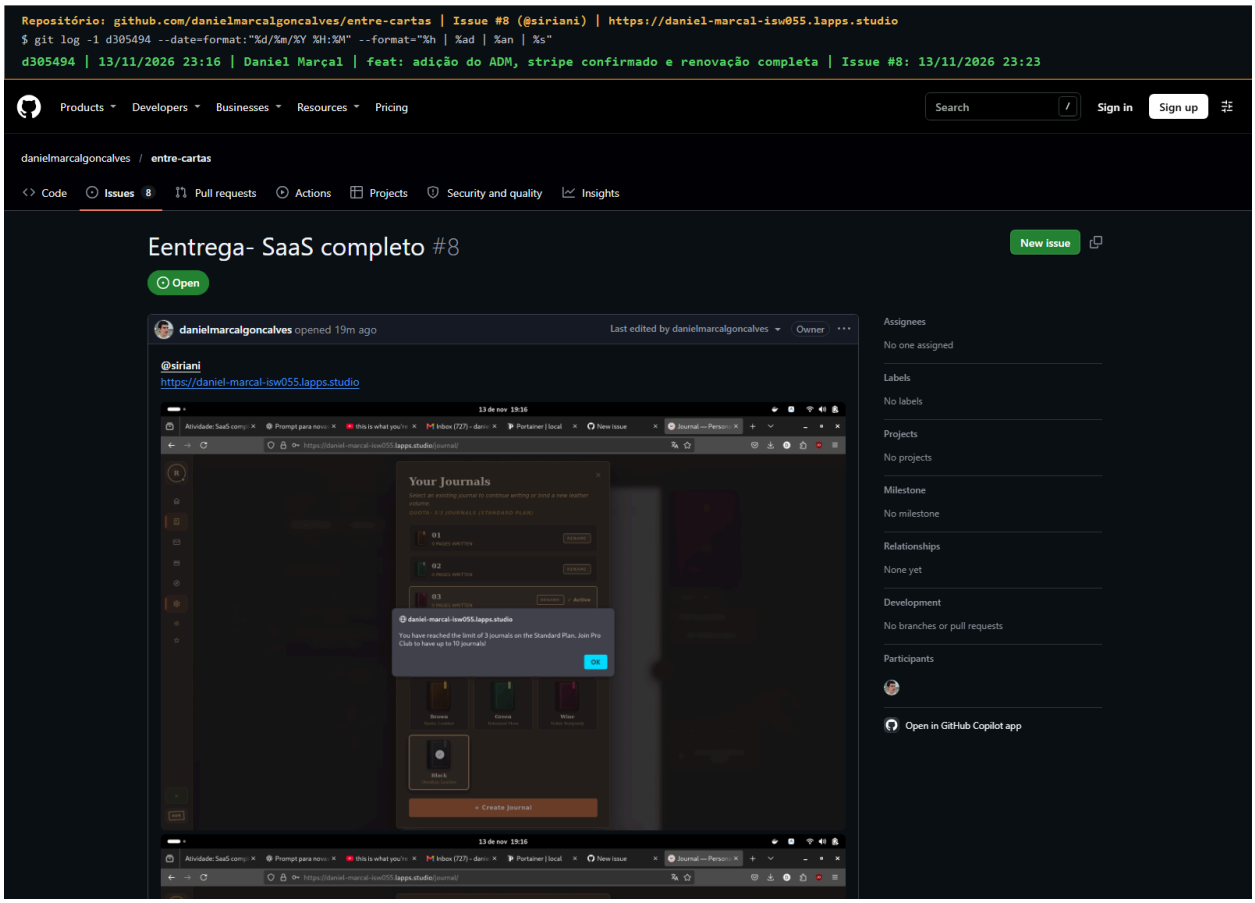


Figura 13: Atividade 11 — Evidência da entrega: Issue 8 mencionando `@siriani` e commit d305494 em 13/11/2026 às 23:16 no GitHub

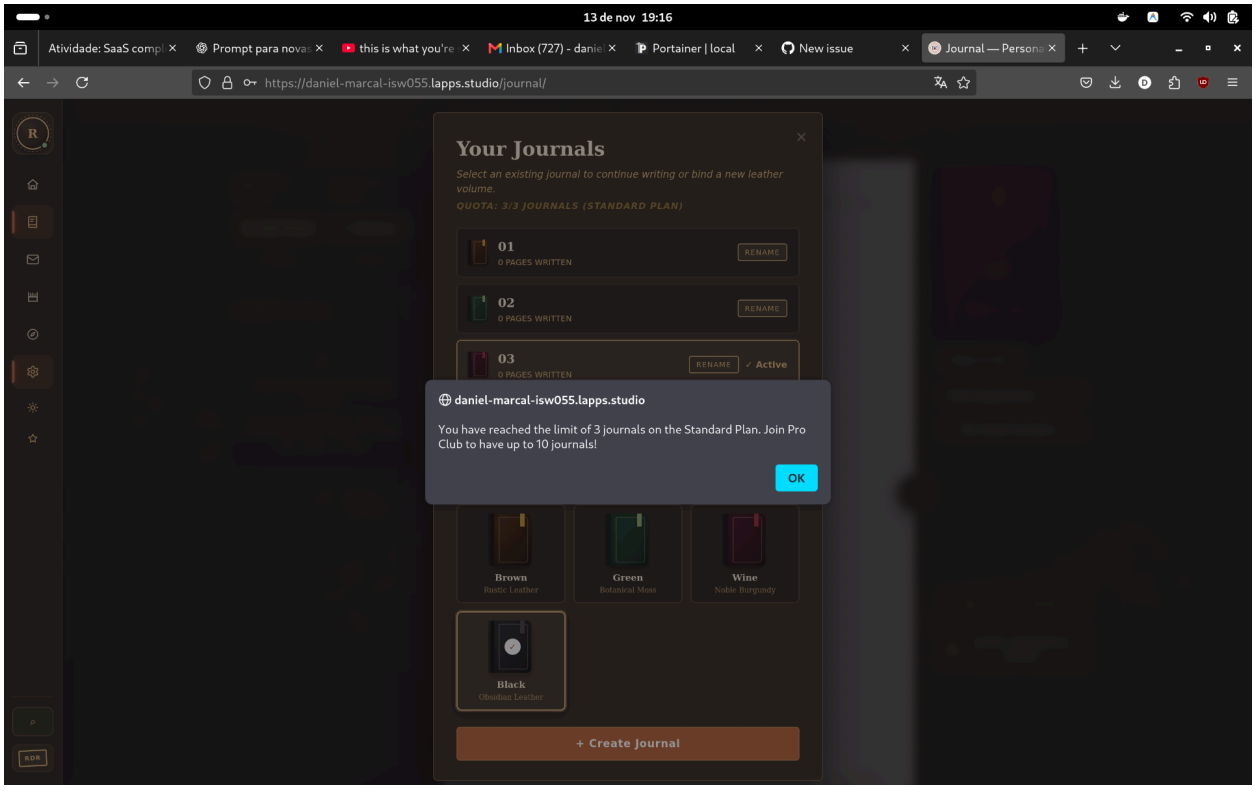


Figura 14: Atividade 11 — Resultado (1/4): Bloqueio real de cota no Plano Standard Free ao tentar criar o 4º caderno em https://daniel-marcal-isw055.lapps.studio/journal

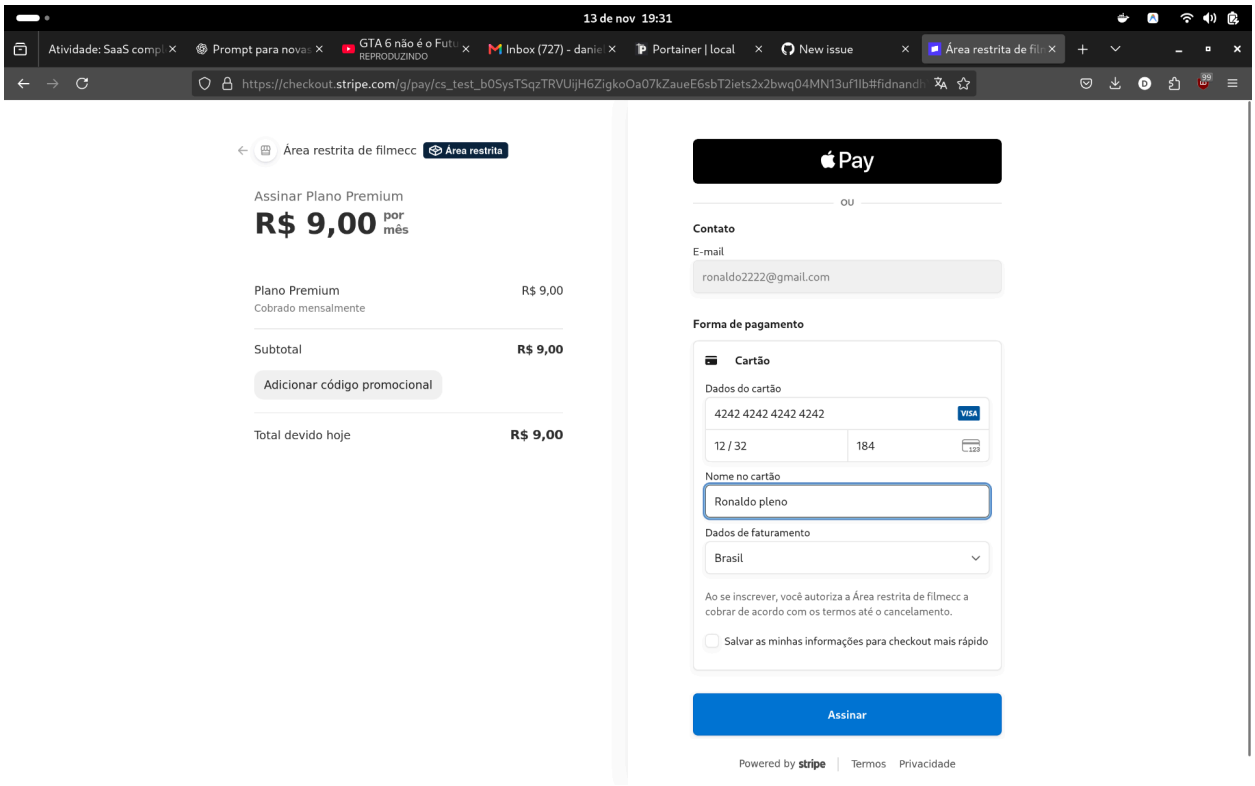


Figura 15: Atividade 11 — Resultado (2/4): Pagamento da assinatura recorrente no Stripe Checkout em modo de teste utilizando o cartão 4242 4242 4242 4242

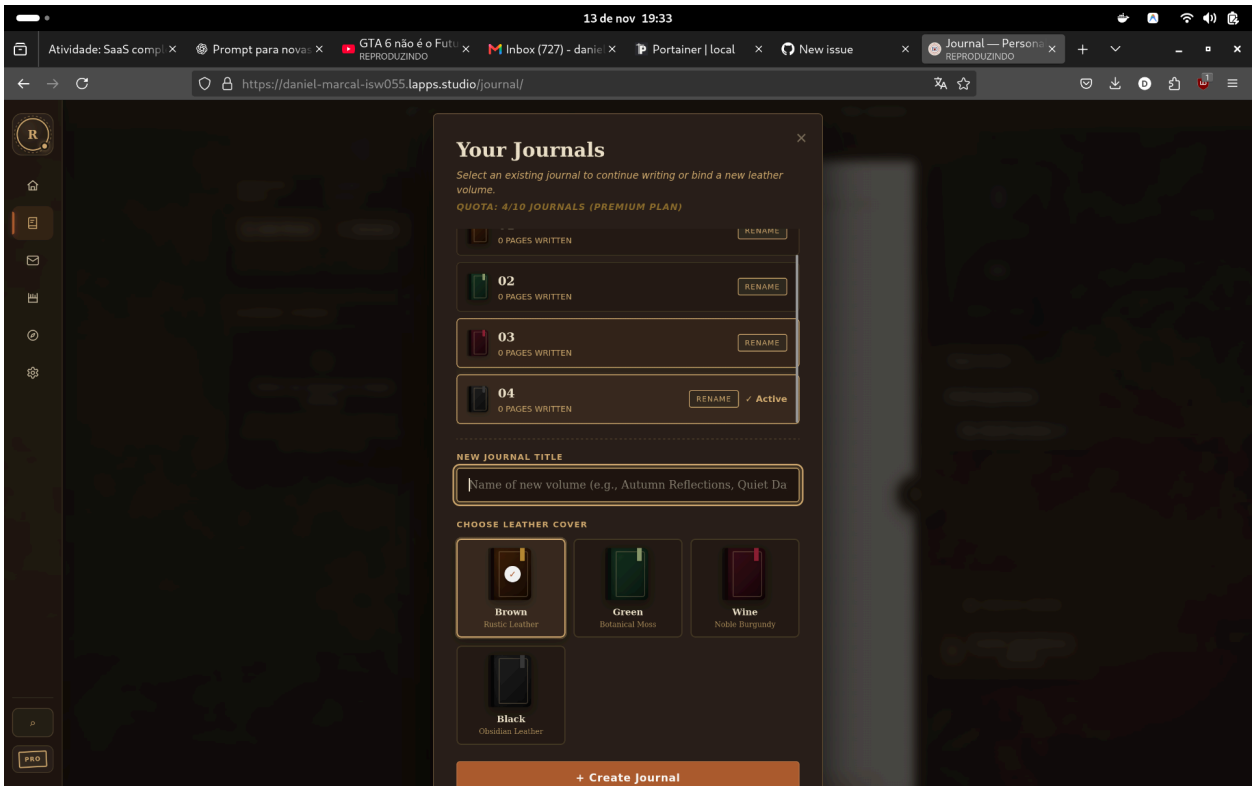


Figura 16: Atividade 11 — Resultado (3/4): Assinatura confirmada via webhook em /clube-pro e cota expandida liberada para 4/10 cadernos no Plano Premium

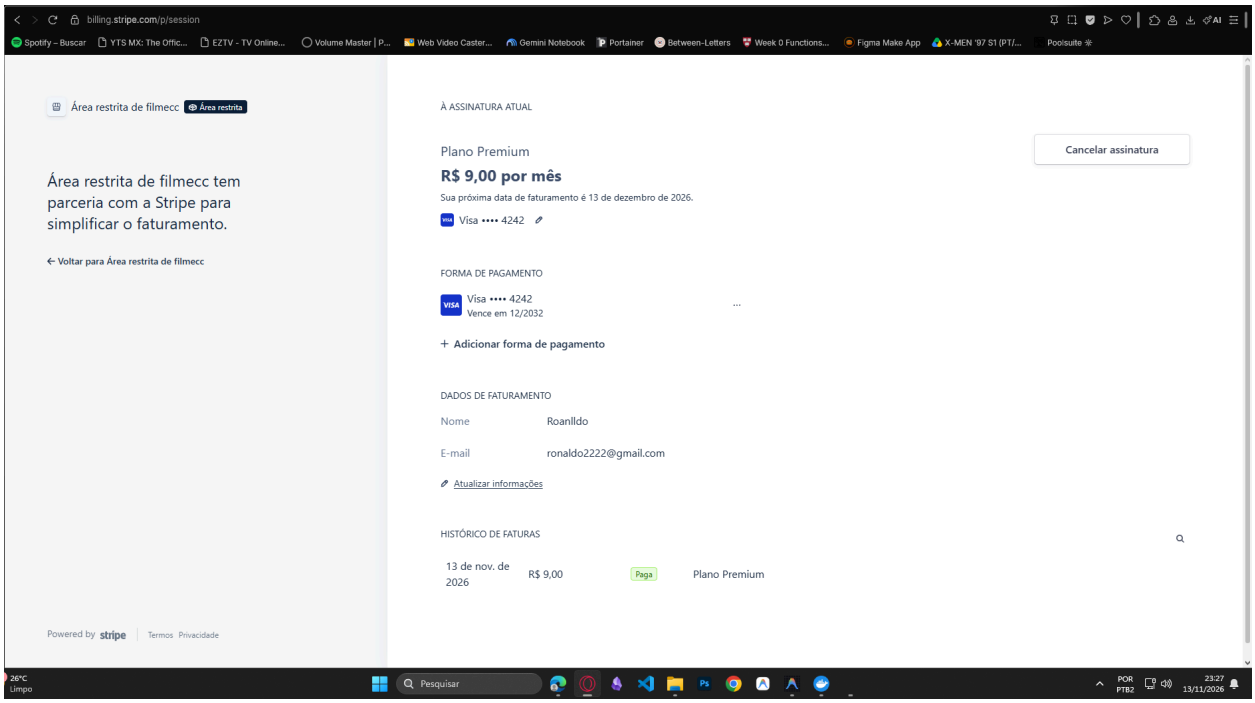


Figura 17: Atividade 11 — Resultado (4/4): Portal do Cliente da Stripe (billing.stripe.com) aberto via POST /api/stripe/portal e Painel Admin com logs de auditoria e bloqueio 403

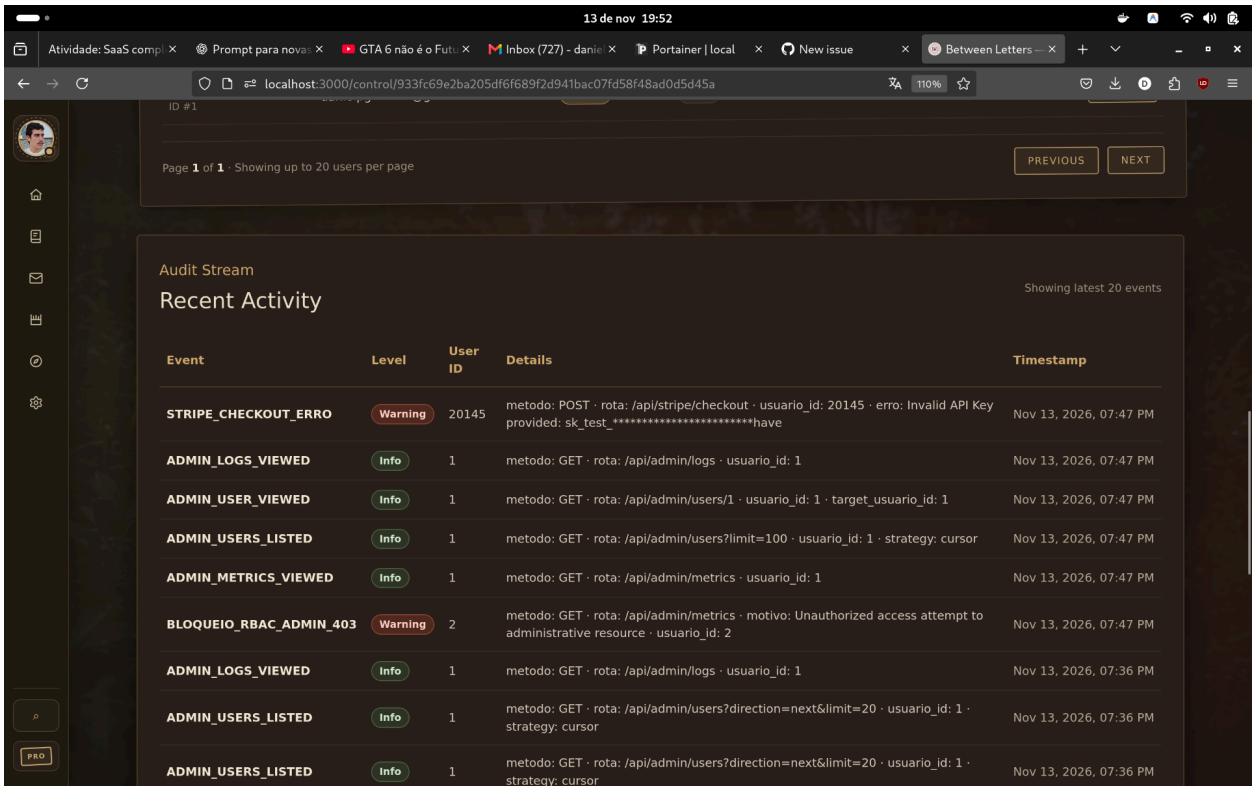


Figura 18: Atividade 11 — Auditoria e Segurança: Registro de eventos administrativos e bloqueio BLOQUEIO_RBAC_ADMIN_403 no stream de auditoria

Dificuldades e como foram resolvidas. Dois desafios críticos surgiram na integração final em produção no Portainer: (1) a rota `/api/stripe` estava originalmente montada antes do `cookieParser()` no `server.js` para preservar o corpo bruto do webhook, o que impedia a leitura do cookie `HttpOnly access_token` nas rotas `/api/stripe/status` e `/api/stripe/portal`; e (2) redirecionamentos da Stripe em contêineres atrás de proxy reverso podiam cair em `localhost` se `APP_URL` não estivesse definido.

Ambos foram resolvidos movendo `cors` e `cookieParser` para antes de `/api/stripe` (mantendo apenas `express.json()` depois) e criando o utilitário `resolvePublicAppUrl(req)`, que deduz automaticamente o domínio público real a partir dos cabeçalhos `X-Forwarded-Proto` e `X-Forwarded-Host`.

6 Considerações finais

A construção do **Entre Cartas (Between Letters)** ao longo deste segundo bimestre consolidou, na prática, a diferença entre programar funcionalidades isoladas e arquitetar um produto SaaS completo em nuvem. Vivenciar o ciclo integral — desde a modelagem do plano de negócios e do **Lean Canvas**, passando pela orquestração de seis contêineres interdependentes no Portainer, até a implementação de webhooks financeiros idempotentes, controle de cotas por plano, armazenamento de objetos no MinIO e telemetria administrativa sobre Redis Streams — proporcionou um domínio concreto sobre como sistemas comerciais modernos operam com segurança e previsibilidade.

Cabe ressaltar, mais uma vez, a complexidade técnica e o potencial real de escalabilidade atingidos pelo nosso projeto. Embora algumas etapas intermediárias tenham sido entregues após a data inicialmente planejada, esse tempo adicional foi integralmente revertido na construção de uma plataforma robusta, com acabamento visual autoral, motor próprio de paginação e **scrapbook** interativo, isolamento rigoroso de dados multi-usuário e arquitetura **stateless** nos serviços de aplicação (`app`, `auth-service` e `log-service`). Graças a esse desenho desacoplado, o **Entre Cartas** pode escalar horizontalmente com facilidade — bastando adicionar réplicas dos contêineres Node.js atrás do balanceador/proxy ou migrar o MariaDB, o Redis e o MinIO para serviços gerenciados em nuvem pública —, cumprindo com maestria a proposta da disciplina de criar um SaaS real pronto para comercialização.

Como próximos passos para a evolução comercial do produto em direção ao lançamento para o público geral, planeja-se: (1) substituir as chaves de teste (`sk_test_...`) pelas chaves de produção da Stripe após homologação fiscal; (2) adicionar exportação de cadernos e cartas completas em formato PDF editorial de alta resolução como benefício adicional do **Clube do Remetente Pro**; e (3) implementar uma esteira automatizada de CI/CD via GitHub Actions com testes ponta a ponta antes de cada atualização da stack em produção.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Daniel Marçal Gonçalves

Pompeia, 13/11/2026